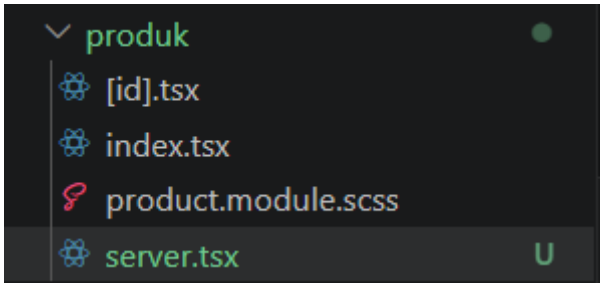
KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah	:	Pemograman Framework
Program Studi	:	D4 – Teknik Informatika
Semester	:	6

Kelas	:	TI – 3B
NIM	:	2341720169
Nama	:	Dimas Adi Bayu S
Jobsheet ke-	:	9

Laporan Jobsheet

Praktikum 1

Langkah	Jawaban/Deskripsi
1	Buat file baru pada pages/products/server.tsx 
2	Modifikasi file server.tsx :

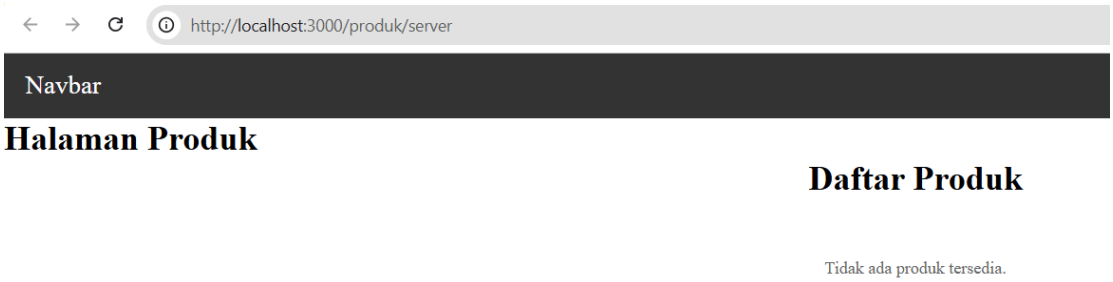
<table>
<tr><td></td><td>

src > pages > produk > server.tsx > HalamanProdukServer

```
import TampilanProduk from "../views/product";

Windsurf: Refactor | Explain | Generate JSDoc | X
const HalamanProdukServer = () => {
  return (
    <div>
      <h1>Halaman Produk</h1>
      <TampilanProduk products={[]} />
    </div>
  )
};

export default HalamanProdukServer;
```

</td></tr>
<tr><td>3</td><td>

Jalankan browser : http://localhost:3000/produk/server



</td></tr>
</table>

Praktikum Ke-2

Langkah	Jawaban/Deskripsi
1	Implementasi getServerSideProps pada server.tsx

<table>
<tr>
<td></td>
<td>

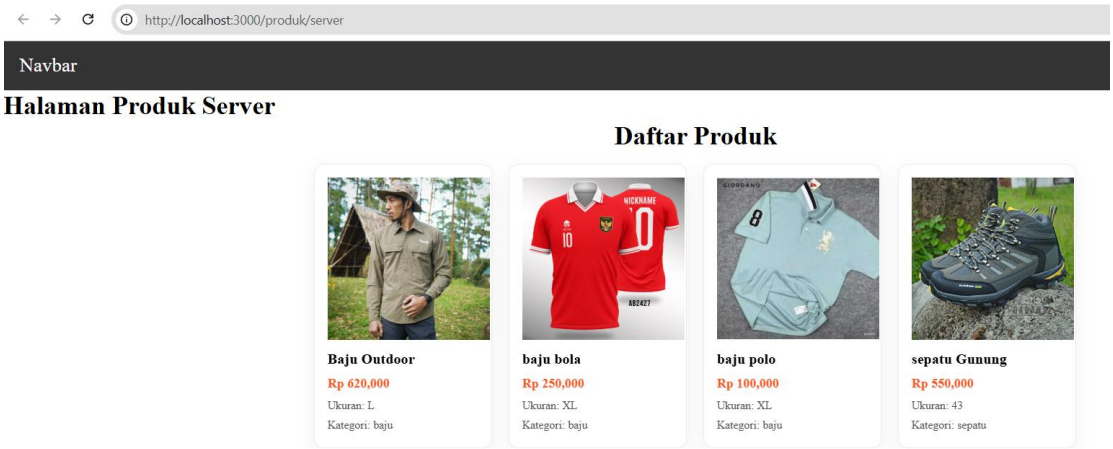
src > pages > produk > server.tsx > getServerSideProps

```
import { exp } from "firebase/firestore/pipelines";
import TampilanProduk from "../views/product";

Windsurf: Refactor | Explain
type ProductType = {
    id: string;
    name: string;
    price: number;
    size: string;
    category: string;
    image: string;
}

Windsurf: Refactor | Explain | Generate JSDoc | ×
const HalamanProdukServer = (props:{products:ProductType[]}) => {
    const { products } = props;
  return (
    <div>
      <h1>Halaman Produk</h1>
      <TampilanProduk products={products} />
    </div>
  )
};

export default HalamanProdukServer;
```

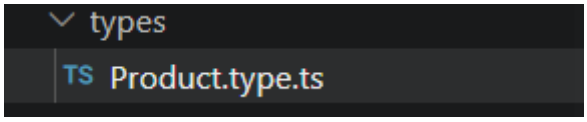
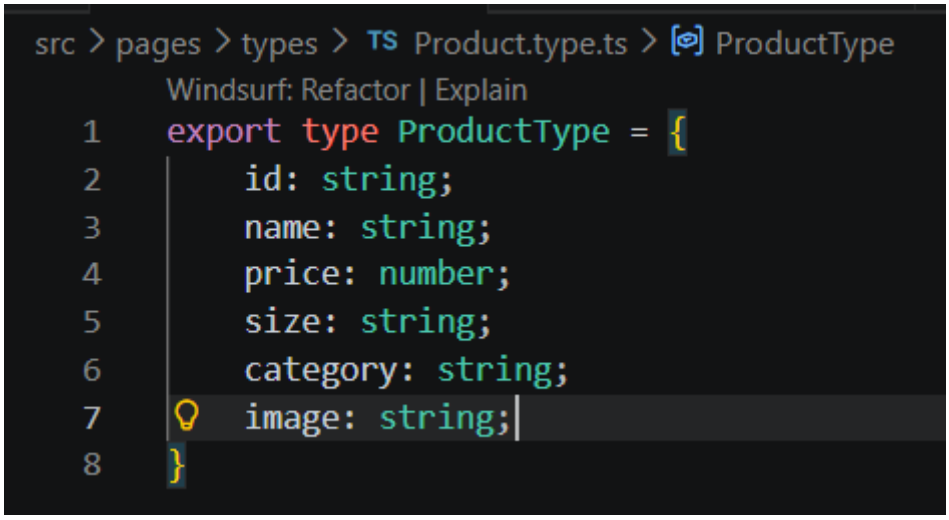
Windsurf: Refactor | Explain | Generate JSDoc | ×

```
export async function getServerSideProps() {
    const res = await fetch("http://localhost:3000/api/products");
    const data = await res.json();

    return {
        props: {
            products: data,
        },
    };
}
```

</td>
</tr>
<tr>
<td>2</td>
<td>Jalankan browser http://localhost:3000/produk/server</td>
</tr>
</table>

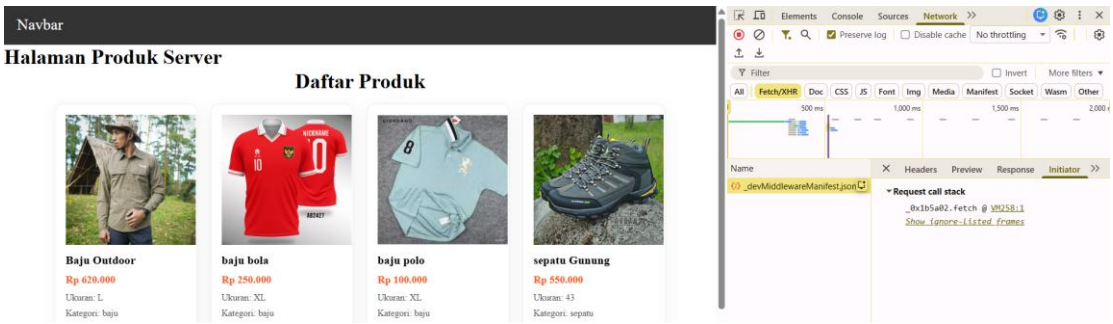
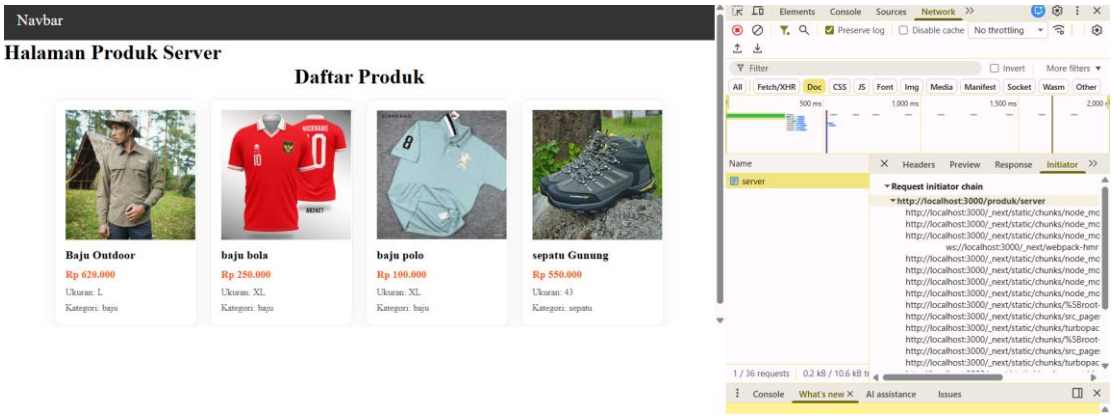
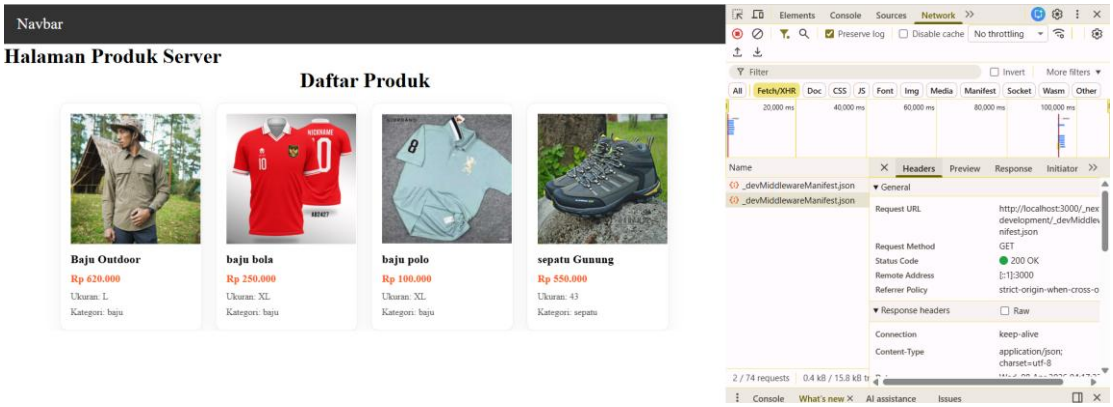
Praktikum Ke-3

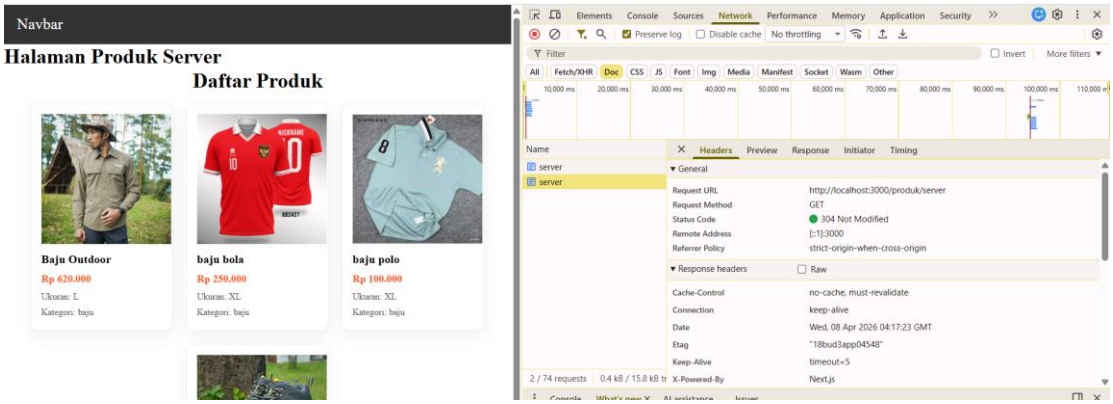
Langkah	Jawaban/Deskripsi
1	Buat folder types pada folder pages dan buat file Product.type.ts 
2	Modifikasi Product.type.ts 
3	Setelah membuat file Product.type.ts maka modifikasi pada file server.tsx menjadi

	src > pages > produk > server.tsx > getServerSideProps
	Windsurf: Refactor | Explain | Generate JSDoc | X
	`const HalamanProdukServer = (props:{products:ProductType[]}) => {`
	`  const { products } = props;`
	`  return (`
	`  <div>`
	`  <h1>Halaman Produk Server</h1>`
	`  <TampilanProduk products={products} />`
	`  </div>`
	`  )`
	`};`
	
	`export default HalamanProdukServer;`
	
	Windsurf: Refactor | Explain | Generate JSDoc | X
	`export async function getServerSideProps() {`
	`  const res = await fetch("http://localhost:3000/api/produk");`
	`  const response = await res.json();`
	
	`  return {`
	`  props: {`
	`  products: response.data,`
	`  },`
	`  };`
	`}`

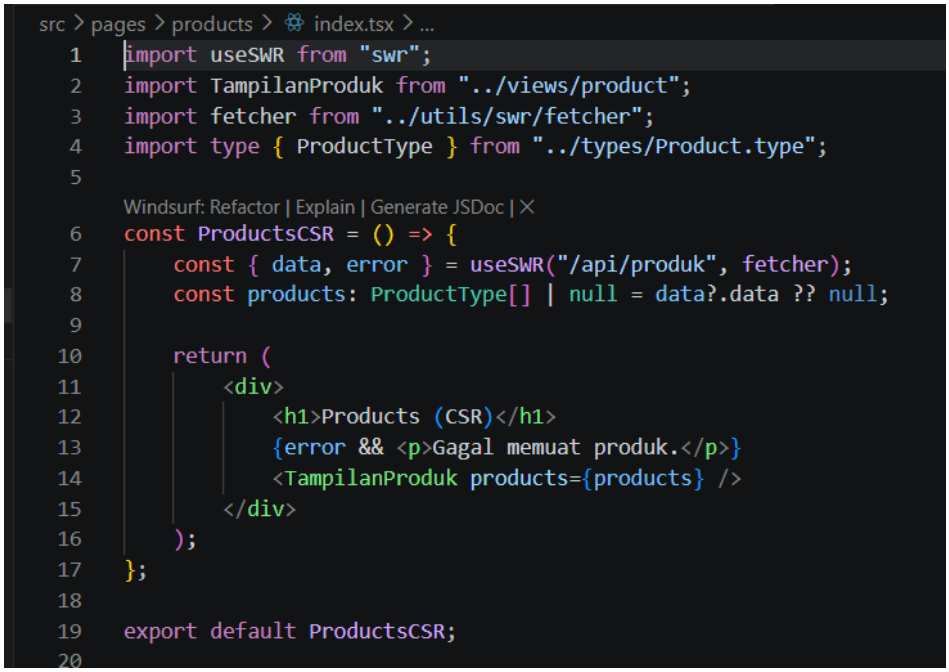
Praktikum Ke-4

Langkah	Jawaban/Deskripsi
1	Uji 1 – Skeleton • Buka halaman CSR • Refresh • Skeleton muncul • Buka halaman SSR • Refresh • Skeleton tidak muncul

	 
2	Uji 2 – Network Tab 1. Buka DevTools → Network → XHR 2. Refresh halaman CSR → Request API terlihat  Refresh halaman SSR → Request API tidak terlihat

Tugas

Langkah	Jawaban/Deskripsi
1	Buat 2 halaman: - /products (CSR) 

```
src > pages > products > index.tsx > ...
import useSWR from "swr";
import TampilanProduk from "../views/product";
import fetcher from "../utils/swr/fetcher";
import type { ProductType } from "../types/Product.type";

Windsurf: Refactor | Explain | Generate JSDoc | X
const ProductsCSR = () => {
    const { data, error } = useSWR("/api/produk", fetcher);
    const products: ProductType[] | null = data?.data ?? null;

    return (
        <div>
            <h1>Products (CSR)</h1>
            {error && <p>Gagal memuat produk.</p>}
            <TampilanProduk products={products} />
        </div>
    );
};

export default ProductsCSR;
```

	- /products/server (SSR)

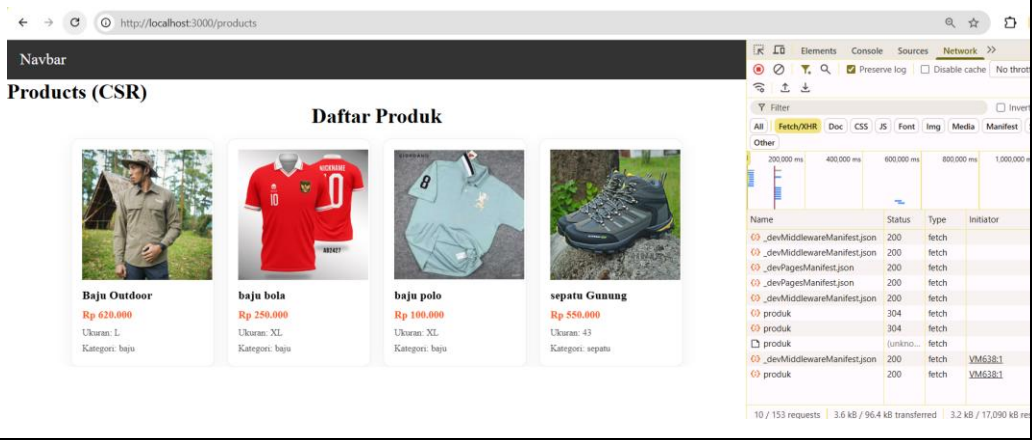
src > pages > products > server.tsx > ...

```
import type { GetServerSideProps } from "next";
import TampilanProduk from "../views/product";
import type { ProductType } from "../types/Product.type";

Windsurf: Refactor | Explain
type Props = {
    products: ProductType[];
};

Windsurf: Refactor | Explain | Generate JSDoc | ×
const ProductsServer = ({ products }: Props) => {
    return (
        <div>
            <h1>Products (SSR)</h1>
            <TampilanProduk products={products} />
        </div>
    );
};

Windsurf: Refactor | Explain | Generate JSDoc | ×
export const getServerSideProps: GetServerSideProps<Props> = async ()
    const res = await fetch("http://localhost:3000/api/produk");
    const response = await res.json();

    return {
        props: {
            products: response?.data ?? [],
        },
    };
};

export default ProductsServer;
```

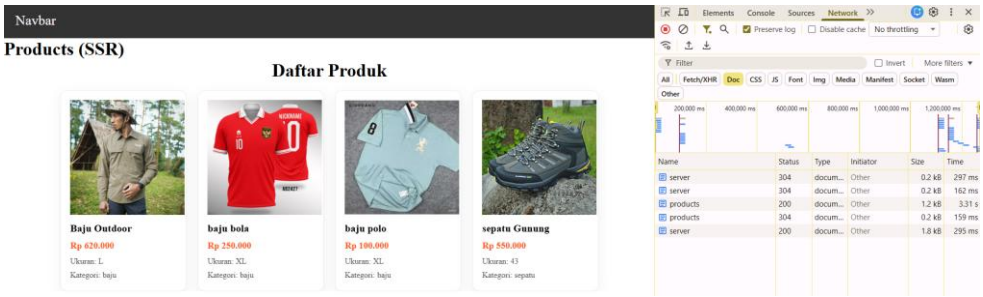
2

Dokumentasikan:

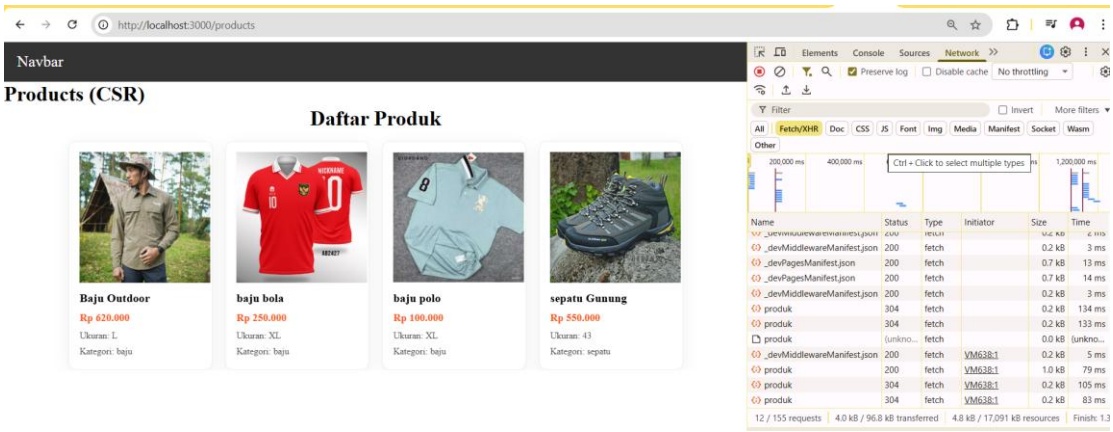
- Screenshot CSR

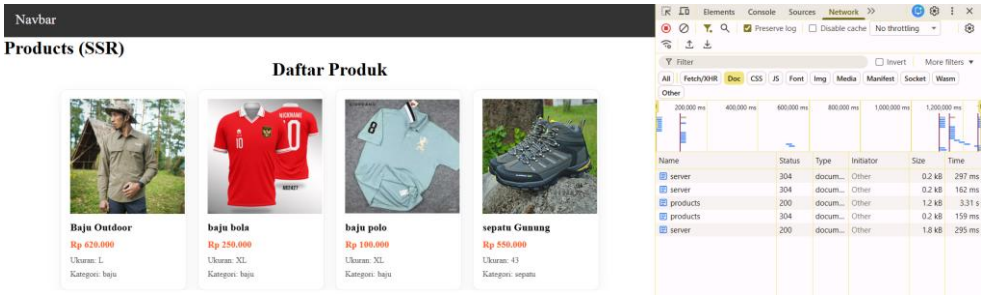


- Screenshot SSR

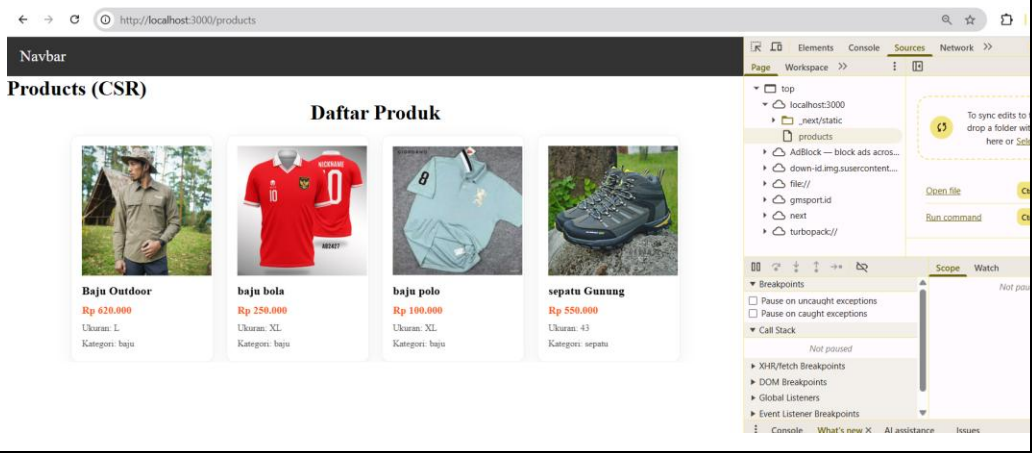


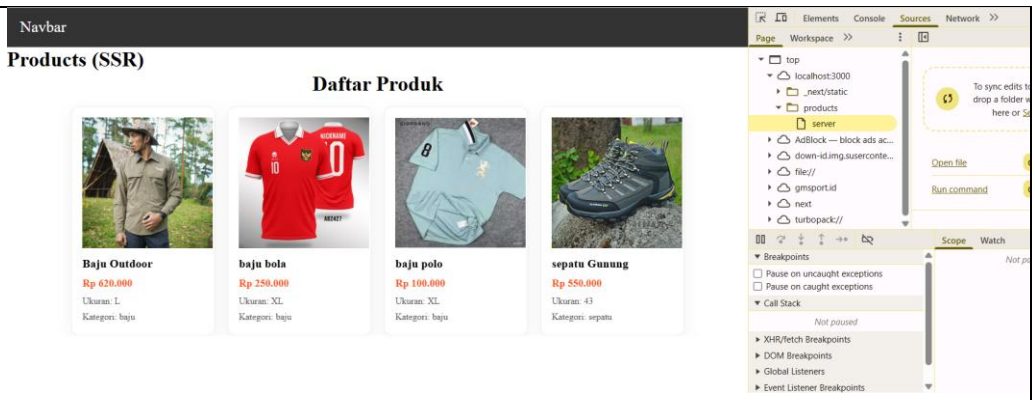
- Perbedaan Network tab





- Perbedaan View Source



	
3	Buat laporan analisis minimal 2 halaman. Berdasarkan praktikum yang telah dilaksanakan, Next.js terbukti bukan sekadar framework frontend, melainkan framework fullstack karena kemampuannya menangani sisi klien sekaligus sisi server melalui fitur API Routes. Pada bagian backend, API Routes yang diletakkan di dalam folder pages/api/ memungkinkan pengembang untuk membuat endpoint yang mengirimkan respons data dalam format JSON dengan status code tertentu. Dalam implementasi ini, file produk.ts digunakan untuk menyediakan data produk yang kemudian dikonsumsi oleh frontend menggunakan fungsi fetch.Analisis mendalam dilakukan pada perbedaan metode rendering data, yaitu antara data statis dan dinamis. Data statis adalah data yang ditulis langsung di dalam kode program (hardcoded) dan bersifat tetap , sementara data dinamis diambil dari sumber eksternal seperti Firebase Firestore yang memungkinkan perubahan isi data secara real-time tanpa perlu mengubah kode sumber aplikasi. Penggunaan Firebase sebagai database NoSQL sangat mendukung aplikasi modern karena kemudahan integrasinya melalui SDK Firebase yang diinstal melalui NPM.Keamanan aplikasi menjadi poin krusial dalam analisis ini, terutama pada pengelolaan environment variable dalam file .env.local. Sangat penting untuk tidak melakukan push file .env.local ke repositori publik karena file tersebut berisi kredensial sensitif seperti API Key dan Project ID Firebase. Jika data rahasia ini terekspos, pihak yang tidak bertanggung jawab dapat menyalahgunakan akses ke database.Dalam aspek rendering, terdapat perbedaan signifikan antara Client-Side Rendering (CSR) dan Server-Side Rendering (SSR). Pada metode CSR (seperti yang digunakan pada langkah awal praktikum), pengambilan data dilakukan di browser

	menggunakan useEffect. Hal ini memungkinkan munculnya skeleton screen sebagai indikator pemuatan sementara browser menunggu data dari API. Sebaliknya, pada SSR, halaman dikirimkan ke browser dalam bentuk HTML utuh yang sudah berisi konten lengkap. Hal ini membuat SSR jauh lebih unggul untuk SEO karena mesin pencari dapat langsung mengindeks konten tanpa harus menjalankan JavaScript terlebih dahulu.Secara keseluruhan, integrasi antara Next.js sebagai framework fullstack dengan Firebase Firestore memberikan solusi pengembangan aplikasi yang efisien. Mahasiswa tidak hanya belajar menampilkan data dinamis ke halaman web , tetapi juga memahami arsitektur komunikasi data yang aman dan optimal untuk performa serta visibilitas mesin pencari

Pertanyaan

Langkah	Jawaban/Deskripsi
1	Jawab pertanyaan berikut: 1. Mengapa SSR lebih baik untuk SEO? 2. Kapan sebaiknya menggunakan SSR? 3. Apa kekurangan SSR dibanding CSR? 4. Mengapa skeleton tidak muncul pada SSR? Jawab : 1. Karena konten sudah dalam bentuk HTML utuh saat sampai di browser, sehingga Google Bot bisa langsung membaca isi halaman tanpa harus menjalankan JavaScript terlebih dahulu. 2. Gunakan SSR untuk halaman yang isinya sering berubah (dinamis) tetapi butuh ditemukan di pencarian Google (SEO), seperti halaman detail produk atau portal berita. 3. a. Server bekerja lebih keras karena harus menyusun halaman untuk setiap pengunjung. b. Klik pertama terasa sedikit lebih lama karena browser harus menunggu server selesai "memasak" halaman.

	4. Karena browser baru menerima tampilan setelah datanya lengkap. Berbeda dengan CSR yang mengirim "kerangka" (skeleton) dulu baru mengisi datanya kemudian.